



Cahier de projet

Le pipeline de commandes

LivrExpress

Stockage objet, base NoSQL et files de messages : concevoir et réaliser, en local, une architecture de données de type AWS.

Services étudiés : Amazon S3 (MinIO) · Amazon DynamoDB (DynamoDB Local) · Amazon SQS (ElasticMQ)

Langage du projet : Go

Nom : Prénom :
Groupe :

Document élève — à compléter et à rendre avec le code du projet.

1. Contexte

« Vendredi noir, 21 h 14. Le site de LivrExpress reçoit quarante commandes par seconde. À 21 h 15, le serveur de base de données sature. À 21 h 20, le site répond “Erreur 500”. Quand tout redémarre, 1 312 commandes payées ont disparu, et personne ne sait lesquelles. »

LivrExpress est une entreprise algérienne de livraison de colis (Alger, Oran, Constantine, Annaba...). En cinq ans, elle est passée de 200 à 30 000 commandes par jour. Son système d'information, lui, n'a pas changé :

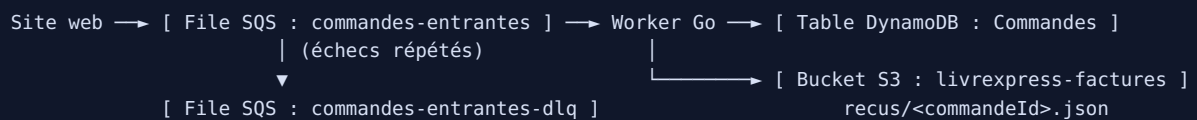
- un **serveur unique** héberge le site web, la base MySQL et les fichiers ;
- les **factures PDF et bons de livraison** sont enregistrés sur un partage réseau qui se remplit chaque mois ;
- le site **écrit directement** chaque commande en base : si la base ralentit, le site ralentit ; si elle tombe, les commandes sont perdues ;
- une facture écrasée par erreur est **définitivement perdue** ; il n'existe pas d'historique.

Après l'incident, Karim (directeur technique) décide de reconstruire la chaîne de traitement des commandes sur des services de données managés, sur le modèle d'AWS. Vous rejoignez l'équipe comme **ingénieur·e cloud junior**, aux côtés de Yasmine (données) et Mehdi (exploitation).

1.1 Le problème à résoudre

Problème constaté	Besoin exprimé par l'entreprise
Le partage de fichiers sature	Un stockage de documents sans limite pratique de capacité, organisé, avec nettoyage automatique des fichiers temporaires.
Factures écrasées ou supprimées par erreur	Pouvoir revenir à une version précédente d'un document.
Envoi de factures aux clients par e-mail en pièce jointe	Partager un document précis, pour une durée limitée, sans rendre le stockage public.
La base relationnelle unique ne suit plus la charge	Une base qui répond en temps constant quel que soit le volume, pour des accès simples et connus à l'avance.
Commandes perdues quand la base est indisponible	Ne jamais perdre une commande acceptée, même si le traitement est en panne.
Un message mal formé bloque tout le traitement	Isoler les messages défectueux sans arrêter la chaîne.

1.2 L'architecture cible



Vous ne disposez d'aucun compte AWS : tout est **émulé sur votre poste** avec des logiciels qui parlent les mêmes API. Ce que vous apprenez ici se transpose tel quel sur AWS : seule l'adresse du service (l'*endpoint*) change.

Service AWS	Émulateur local	Adresse de l'API	Interface web
Amazon S3	MinIO	<code>http://127.0.0.1:9100</code>	<code>http://127.0.0.1:9101</code>
Amazon DynamoDB	DynamoDB Local	<code>http://127.0.0.1:8100</code>	—
Amazon SQS	ElasticMQ	<code>http://127.0.0.1:9324</code>	<code>http://127.0.0.1:9325</code>

Identifiants locaux : clé d'accès `cloudquest` , clé secrète `cloudquest-secret` , région `us-east-1` .

2. Organisation du travail

2.1 Mise en route

```
docker compose up -d          # démarre MinIO, DynamoDB Local et ElasticMQ
go run ./cmd/cloudquest       # démarre le jeu
# puis ouvrir http://127.0.0.1:8090
```

Deux outils en ligne de commande sont fournis, déjà configurés :

```
docker compose run --rm mc ls local
docker compose run --rm aws s3 ls --endpoint-url http://127.0.0.1:9100
```

2.2 Déroulement

Phase	Chapitre CloudQuest	Ce que vous construisez	Durée indicative
1	L'entrepôt infini (S3)	Buckets, objets, préfixes, métadonnées, versioning, cycle de vie, URL pré-signée, politique de bucket	3 h
2	Le registre éclair (DynamoDB)	Tables, clés simples et composées, index secondaire, requêtes, TTL	3 h
3	Le tapis roulant (SQS)	Files, envoi et réception, délai de visibilité, DLQ, file FIFO	2 h 30
4	Projet final	Le worker Go qui relie les trois services, puis sa résistance aux données toxiques	4 h

Règle du jeu. Chaque défi de l'application propose un *parcours guidé* : une explication de ce qu'il faut obtenir, puis des indices du plus vague au plus précis. Un indice coûte 10 % des points du défi (plancher : 40 %). Aucun indice ne donne la commande complète : la solution, c'est vous qui l'écrivez. La vérification interroge les vrais services, les essais sont illimités.

Ce qui est évalué. Pas seulement « ça marche ». Vous devez pouvoir *expliquer* chaque choix : c'est l'objet des questions de ce cahier. Une réponse copiée sans justification ne rapporte aucun point.

2.3 Méthode conseillée pour chaque défi

1. **Lire** le récit et l'objectif : que doit-on observer à la fin ?

2. **Chercher** dans la documentation officielle liée au défi (colonne de droite) le nom de l'opération utile.
3. **Essayer**, lire le message d'erreur en entier, corriger.
4. **Vérifier** dans l'application ; si un point est rouge, son détail indique quoi regarder.
5. En dernier recours, **révéler un indice**, un seul à la fois.
6. **Noter** dans ce cahier la commande ou le code retenu et ce que vous avez compris.

3. Phase 1 — Stockage objet (S3 / MinIO)

Objectif : remplacer le partage de fichiers par un stockage objet. À la fin de la phase, le bucket `livrexpress-factures` existe, est organisé par préfixes et protégé par le versioning.

1 Comprendre le modèle

Avant toute commande, repérez dans la documentation ce qui distingue un *bucket*, un *objet* et une *clé*. Créez votre premier bucket avec l'outil de votre choix, puis observez-le avec un **autre** outil.

Piste : l'AWS CLI contacte le vrai AWS si on ne lui précise pas où se trouve le service.

2 Organiser et décrire

Déposez plusieurs factures sous une arborescence par année et par mois, puis listez uniquement celles d'un mois donné. Ajoutez à un objet un type de contenu et une métadonnée personnalisée.

Piste : comparez le résultat d'un listage avec et sans délimiteur.

3 Protéger et partager

Activez le versioning, provoquez un « accident », restaurez. Programmez le nettoyage automatique d'un préfixe temporaire. Générez un lien de téléchargement valable quelques minutes.

Piste : après une suppression, listez les *versions* de l'objet, pas seulement les objets.

Questions — phase 1

Q1. Un collègue affirme : « S3, c'est un disque dur dans le cloud. » Donnez deux différences concrètes entre un système de fichiers et un stockage objet.

Q2. Vous avez listé les objets du préfixe `factures/2026/10/`. Ce « dossier » existe-t-il réellement ? Justifiez à partir de ce que vous avez observé.

Q3. Après la suppression d'une facture dans un bucket versionné, combien de versions voyez-vous et de quelle nature ? Que faudrait-il faire pour la supprimer définitivement ?

.....

.....

.....

Q4. Le service client veut envoyer une facture à un client. Comparez trois solutions : pièce jointe, bucket public, URL pré-signée. Laquelle recommandez-vous et pourquoi ?

.....

.....

.....

Q5. Le versioning conserve toutes les anciennes versions. Quel est le risque à long terme et quel mécanisme vu dans cette phase permet de le maîtriser ?

.....

.....

4. Phase 2 — Base NoSQL (DynamoDB)

Objectif : stocker les commandes dans une table dont les temps de réponse ne dépendent pas du volume. À la fin de la phase, la table `Commandes` existe et vous savez l'interroger efficacement.

Changement de méthode. En SQL, on modélise les données puis on écrit les requêtes. En DynamoDB, on fait l'inverse : on liste d'abord les accès dont l'application a besoin, puis on choisit les clés qui les rendent possibles.

1 Lister les accès

Avant de créer quoi que ce soit, complétez ce tableau pour LivrExpress.

Besoin de l'application	Table	Clé(s) utilisée(s)
Retrouver une commande par son identifiant		
Lister toutes les étapes de livraison d'un colis, dans l'ordre		
Lister les commandes en attente		

2 Créer, écrire, lire

Créez la table, insérez quelques commandes, relisez-en une par sa clé. Observez le format JSON « typé » des attributs.

Piste : seuls les attributs de clé se déclarent à la création ; les autres sont libres.

3 Interroger sans tout parcourir

Répondez à une question métier de deux façons : en parcourant toute la table, puis en ciblant une partition. Comparez le nombre d'éléments *examinés* et *renvoyés*.

Piste : la réponse de l'API contient deux compteurs distincts.

Questions — phase 2

Q6. Expliquez avec vos mots la différence entre `Query` et `Scan`. Sur une table de 50 millions d'éléments, laquelle est acceptable en production et pourquoi ?

.....

.....

Q7. Pourquoi la table des livraisons a-t-elle besoin d'une clé de tri alors que la table des commandes n'en a pas ?

.....

.....

.....

Q8. Vous avez créé un index secondaire global sur le statut. Que coûte un index (stockage, écritures) ? Le statut est-il un bon choix de clé de partition si 95 % des commandes sont « LIVREE » ?

.....

.....

.....

Q9. Citez un besoin de LivrExpress pour lequel une base relationnelle resterait un meilleur choix que DynamoDB. Justifiez.

.....

.....

.....

Q10. À quoi sert le TTL sur la table des sessions ? L'expiration est-elle immédiate ? Quelle précaution cela impose-t-il au code qui lit la table ?

.....

.....

5. Phase 3 — Files de messages (SQS)

Objectif : découpler le site web du traitement. À la fin de la phase, la file `commandes-entrantes` existe et vous maîtrisez le cycle recevoir → traiter → supprimer.

1 Envoyer, recevoir... et recevoir encore

Envoyez un message, recevez-le **sans le supprimer**, puis tentez de le recevoir à nouveau immédiatement, et encore une minute plus tard. Notez ce que vous observez à chaque fois.

2 Isoler les messages toxiques

Créez une file de quarantaine et reliez-la à la file principale. Faites échouer volontairement un message assez de fois pour qu'il y bascule.

Piste : la liaison se configure sur la file *source*, pas sur la file de quarantaine.

3 Garantir l'ordre

Comparez une file standard et une file FIFO en envoyant une séquence numérotée. Observez le rôle du groupe de messages et de l'identifiant de déduplication.

Questions — phase 3

Q11. Reprenez l'incident du vendredi noir. Expliquez précisément en quoi une file entre le site et la base aurait évité la perte des 1 312 commandes.

Q12. Qu'est-ce que le délai de visibilité ? Que se passe-t-il s'il est plus court que la durée du traitement ? Et s'il est beaucoup trop long ?

Q13. Une file standard garantit une livraison « au moins une fois ». Quelle conséquence pour le code du consommateur ?

Q14. Pour les paiements, l'équipe hésite entre file standard et FIFO. Donnez un argument pour chacune, puis tranchez.

6. Phase 4 — Projet final : le worker

« Jour J. Le site va publier de vraies commandes dans la file. Ton programme les lit, les enregistre, dépose un reçu, et ne perd rien. Même si on le coupe au milieu. » — Karim

6.1 Cahier des charges

Le squelette se trouve dans `projet/pipeline/main.go`. Les connexions aux trois services sont écrites ; la boucle principale et l'ordre des opérations aussi. Vous écrivez les fonctions marquées `TODO`.

Réf.	Exigence	Contrôle dans CloudQuest
E1	Chaque message de <code>commandes-entrantes</code> est décodé en une commande (<code>commandeId</code> , <code>client</code> , <code>montant</code> , <code>ville</code>).	Défi « Mise en production du worker »
E2	La commande est écrite dans la table <code>Commandes</code> avec le statut <code>TRAITEE</code> .	
E3	Un reçu JSON contenant au moins <code>commandeId</code> est déposé sous <code>recus/<commandeId>.json</code> dans <code>livrexpress-factures</code> .	
E4	Le message n'est supprimé qu'après le succès de E2 et E3. À la fin, la file est vide.	
E5	Un message dont le corps n'est pas un JSON valide, ou dont le montant est ≤ 0 , n'est ni enregistré ni supprimé.	Défi « Résister aux données toxiques »
E6	<code>commandes-entrantes</code> est reliée à <code>commandes-entrantes-dlq</code> (5 réceptions au plus) ; les messages rejetés y aboutissent.	
E7	Un message défectueux n'interrompt jamais le programme : les commandes valides du même lot sont traitées.	

6.2 Parcours guidé

1 Lire avant d'écrire

Lancez le programme tel quel : `go run ./projet/pipeline`. Que vous dit-il ? Lisez ensuite `main()` et `traiter()` : dessinez sur papier le chemin d'un message, avec les deux sorties possibles (succès, rejet).

2 Recevoir et décoder

Écrivez `recevoir` puis `decoder`. Affichez simplement chaque commande décodée, sans rien écrire ailleurs. Relancez : que deviennent les messages entre deux exécutions ? Pourquoi ?

3 Enregistrer

Écrivez `enregistrer`. Vérifiez dans la table, avec la CLI, qu'une commande apparaît avec tous ses attributs et le bon statut. Lancez le programme deux fois : combien d'éléments pour une même commande ?

4 Déposer le reçu

Écrivez `deposerRecu`. Vérifiez l'objet dans la console MinIO : sa clé, son contenu, son type.

5 Acquitter

Écrivez `acquitter` en dernier. Cliquez « Envoyer les commandes du jour » dans l'application, lancez le worker, vérifiez.

6 Résister

Écrivez `valider`, complétez `rejeter`, branchez la DLQ. Cliquez « Envoyer un lot piégé ». Mesurez le temps que met un message toxique à atteindre la quarantaine, puis cherchez comment le réduire.

Expérience obligatoire — la coupure. Ajoutez temporairement une pause de quelques secondes entre l'écriture DynamoDB et le dépôt S3. Lancez le worker et interrompez-le (`Ctrl+C`) pendant la pause. Relancez-le. Observez l'état de la table, du bucket et de la file, et reportez vos observations à la question Q16.

Questions — phase 4

Q15. Pourquoi l'ordre « enregistrer → déposer le reçu → supprimer le message » est-il imposé ? Que risquerait-on en supprimant le message en premier ?

Q16. Décrivez ce que vous avez observé lors de l'expérience de la coupure. Votre traitement est-il idempotent ? Quel choix de conception le garantit ?

Q17. Pendant les soldes, le volume triple. Que change-t-on dans cette architecture pour suivre la charge ? Qu'est-ce qui ne change pas ?

Q18. Deux messages toxiques sont en DLQ lundi matin. Décrivez la procédure que l'équipe d'exploitation devrait suivre.

Q19. Listez tout ce qu'il faudrait modifier pour faire tourner ce worker sur le vrai AWS (code, configuration, identifiants, droits).

Q20. Synthèse (10 lignes). Reprenez le tableau « Problème constaté / Besoin » de la section 1.1 et indiquez, pour chaque ligne, le service et la fonctionnalité précise qui y répond.

7. Livrables et barème

7.1 À rendre

- Le fichier `projet/pipeline/main.go` complété, qui compile (`go build ./...`).
- Ce cahier, avec les réponses aux questions Q1 à Q20.
- Une capture de votre tableau de bord CloudQuest (grade et points).
- Un court fichier `NOTES.md` : les commandes clés utilisées à chaque phase et les difficultés rencontrées.

7.2 Barème indicatif (sur 20)

Élément	Points	Critères
Défis CloudQuest, chapitres 1 à 3	6	Proportionnel aux points obtenus (les indices révélés réduisent le score).
Projet — exigences E1 à E4	4	Défi « Mise en production » validé ; code lisible, erreurs traitées.
Projet — exigences E5 à E7	3	Défi « Données toxiques » validé ; aucun arrêt brutal du programme.
Questions Q1 à Q20	6	Justesse, justification par l'observation, vocabulaire précis.
Notes et soin du rendu	1	Clarté, commandes reproductibles.

7.3 Pour aller plus loin (facultatif)

- Lancer deux workers en parallèle : les commandes sont-elles traitées en double ? Pourquoi ?
- Écrire un second programme qui vide la DLQ et produit un rapport des messages rejetés dans S3.
- Remplacer l'écriture simple dans DynamoDB par une écriture conditionnelle qui refuse d'écraser une commande déjà traitée.
- Stocker dans l'élément DynamoDB la clé S3 du reçu, puis générer une URL pré-signée à partir d'un `commandeId` .

7.4 Glossaire

Bucket	Conteneur d'objets S3, au nom unique.
Clé / préfixe	Nom complet d'un objet ; début commun à plusieurs clés, utilisé comme « dossier » logique.
Clé de partition / de tri	Attributs qui identifient un élément DynamoDB et déterminent son emplacement et son ordre.
GSI	Index secondaire global : une autre « vue » de la table avec des clés différentes.
Délai de visibilité	Durée pendant laquelle un message reçu est masqué aux autres consommateurs.
DLQ	File de quarantaine recevant les messages qui ont échoué trop de fois.
Idempotence	Propriété d'un traitement que l'on peut rejouer sans changer le résultat.
Endpoint	Adresse réseau de l'API d'un service.